

RESEARCH ARTICLE

Numerical Behavior of the Riemann Zeta Function Using Real-to-Complex Conversion

Jacob Orellana Real

Independent Researcher, Ecuador
jacoboreore@gmail.com

ARTICLE HISTORY

Compiled November 23, 2025

ABSTRACT

The Riemann zeta function $\zeta(s)$ and its zeros lie at the core of modern analytic number theory. On the critical line $\Re(s) = \frac{1}{2}$, the Hardy function

$$Z(t) = e^{i\theta(t)} \zeta\left(\frac{1}{2} + it\right)$$

is real valued and encodes the nontrivial zeros as sign changes. Classical large-scale computations rely heavily on Gram points, Gram intervals, and bracketing strategies for locating zeros.

In this paper we develop a different approach for locating zeros of $Z(t)$ based on a real-to-complex conversion

$$t = \sqrt{N - \frac{1}{4}},$$

which parametrizes large heights t by a real variable N . We combine this parametrization with the Riemann–Siegel formula and Gabcke-type remainder estimates, and introduce a “valley scanner” that detects local minima of $|Z(t)|$ along the resulting trajectory in t -space. Candidate minima are then refined by safeguarded Newton iterations.

We report computations reaching heights up to $t \approx 10^{20}$ and compare our results with existing zero tables, observing excellent agreement in regions with published data and consistent behavior in previously unexplored windows. The method provides an alternative to Gram-point based bracketing that emphasizes the local geometric structure of $Z(t)$ rather than traditional sign-change accounting, and offers a flexible computational tool for further large-scale experiments on the critical line.

KEYWORDS

Riemann zeta function; Hardy Z -function; Riemann–Siegel formula; high-precision computation; numerical analysis; experimental mathematics; real-to-complex mapping; zero-finding algorithms

Primary 11M06; Secondary 11M26, 65B10, 65Y20

1. Introduction

The Riemann zeta function

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s}$$

where $s = \sigma + it$, plays a central role in analytic number theory. Its non-trivial zeros are conjectured to lie entirely on the critical line $\Re(s) = \frac{1}{2}$, the celebrated Riemann Hypothesis (RH).

Modern computational verification of zeros has been dominated by Gram points, the Riemann–Siegel formula, and high-precision approaches such as those developed by Odlyzko. These methods focus on either locating sign changes of $Z(t)$ or numerically determining minima through bracketing techniques.

In this work, we introduce a different approach based on:

1. a **real-to-complex conversion** that maps large integers N to complex values $s = \frac{1}{2} + it$ via a structured transformation, 2. a **valley-scanner algorithm** that identifies zeros as minima of $|Z(t)|$ without requiring Gram points or classical bracketing, 3. high-precision computation enabled through AWS EC2 compute instances, suitable for large-scale exploration.

The method naturally reproduces the symmetry of the critical line and simplifies the evaluation of $Z(t)$ at extremely large heights. We validate the computations by comparing results to known zeros from Odlyzko’s datasets [2]. The framework is reproducible: all datasets, code, and scripts are publicly available through Zenodo and GitHub.

In later sections, we derive the numerical structure of the valley landscape, analyze error propagation in the real-to-complex mapping, and demonstrate stability of the algorithm up to $t \approx 10^{20}$.

2. Mathematical Background

2.1. The Riemann Zeta Function

The Riemann zeta function is defined for complex arguments $s = \sigma + it$ with $\sigma > 1$ by the absolutely convergent series

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s}.$$

Through analytic continuation, this definition extends to all $s \neq 1$. A central identity governing its structure is the functional equation

$$\zeta(s) = 2^s \pi^{s-1} \sin\left(\frac{\pi s}{2}\right) \Gamma(1-s) \zeta(1-s),$$

which reflects values across the critical line. The nontrivial zeros of $\zeta(s)$ lie in the strip $0 < \sigma < 1$. The Riemann Hypothesis (RH) asserts that all such zeros satisfy

$$\Re(s) = \frac{1}{2}.$$

2.2. Relation to Prime Numbers

The zeta function encodes the multiplicative structure of the positive integers via the Euler product

$$\zeta(s) = \prod_{p \text{ prime}} \frac{1}{1 - p^{-s}}, \quad \Re(s) > 1.$$

Its zeros influence the distribution of primes through the explicit formula relating $\pi(x)$ to the nontrivial zeros $\rho = \frac{1}{2} + it_\rho$:

$$\pi(x) \sim \text{Li}(x) - \sum_{\rho} \text{Li}(x^\rho) + (\text{lower-order terms}).$$

This deep connection motivates the search for efficient and accurate methods for computing zeros at large heights.

2.3. The Hardy Function $Z(t)$

On the critical line, the real-valued Hardy function is defined by

$$Z(t) = e^{i\theta(t)} \zeta\left(\frac{1}{2} + it\right),$$

where the Riemann–Siegel theta function is

$$\theta(t) = \arg\left(\Gamma\left(\frac{1}{4} + \frac{it}{2}\right)\right) - \frac{t}{2} \log \pi.$$

The multiplicative factor $e^{i\theta(t)}$ ensures that $Z(t)$ is real for real t . The magnitude $|Z(t)|$ exhibits a distinctive “mountain–valley” landscape, and its zeros correspond precisely to the nontrivial zeros of $\zeta(s)$ on the critical line.

2.4. Average Zero Spacing and Density

The Riemann–von Mangoldt formula states that the number of nontrivial zeros with imaginary part in $(0, T)$ is

$$N(T) = \frac{T}{2\pi} \log\left(\frac{T}{2\pi}\right) - \frac{T}{2\pi} + O(\log T).$$

Thus, the average spacing between consecutive zeros near height T is

$$\Delta T_{\text{avg}} \approx \frac{2\pi}{\log(T/2\pi)}.$$

This estimate provides a meaningful benchmark for assessing the density and distribution of numerically computed zeros.

2.5. Known Computational Approaches

The numerical evaluation of $Z(t)$ and the computation of its zeros have led to a wide range of techniques, from asymptotic expansions to fft-based algorithms.

The Riemann–Siegel Formula

For large t , the Dirichlet series for $\zeta(s)$ converges too slowly to be practical. The Riemann–Siegel formula provides the asymptotic expansion

$$Z(t) = 2 \sum_{n=1}^N \frac{\cos(\theta(t) - t \log n)}{\sqrt{n}} + R(t), \quad N = \left\lfloor \sqrt{\frac{t}{2\pi}} \right\rfloor,$$

with a small remainder $R(t)$. Gabcke’s refinements improve the evaluation of the remainder and ensure stability at large heights.

Gram Points

Gram points are values g_n defined by

$$\theta(g_n) = n\pi.$$

Empirically, zeros of $Z(t)$ tend to alternate with Gram points, although “Gram’s law” fails infinitely often. Nonetheless, Gram points remain a traditional tool for locating sign changes.

Turing’s Method

Turing introduced a verification procedure [4] ensuring that no zeros are missed in an interval by comparing the number of sign changes with the theoretical count predicted by the Riemann–von Mangoldt formula. This remains the standard completeness check in computational work.

Odlyzko–Schönhage Algorithm

At very large heights, the Odlyzko–Schönhage [5] method leverages FFT-based acceleration to evaluate $Z(t)$ at many points simultaneously. This approach enabled computations of millions of zeros far beyond 10^{20} and remains the state-of-the-art for ultra-high-range verification.

Limitations at High t

As t increases:

- the oscillation of $Z(t)$ intensifies, increasing the required working precision;
- the Riemann–Siegel expansion shows cumulative sensitivity to remainder terms;
- FFT-based methods face memory scaling issues and difficulty integrating arbitrary precision.

These constraints motivate alternative formulations aimed at numerical stability and conceptual simplicity—such as the real-to-complex conversion and valley scanning

methods developed in this work.

3. Proposed Numerical Method

3.1. Motivation and Research Shift

Although the present work focuses on the Riemann zeta function and the numerical detection of its zeros, the underlying method originated from an independent study of semiprime numbers. In that earlier investigation, the goal was to analyze the algebraic relationships between a semiprime $N = p_1 p_2$ and its constituent prime factors by examining symmetry structures embedded in associated quadratic forms.

Given

$$N = p_1 p_2, \quad S = p_1 + p_2,$$

the classical quadratic equation

$$x^2 - Sx + N = 0$$

has roots p_1 and p_2 . In standard notation, the solutions are

$$x = \frac{S \pm \sqrt{S^2 - 4N}}{2}.$$

While this framework is algebraically natural, it proved difficult to extend into an efficient computational method for factor analysis and was eventually set aside. Nevertheless, the structure revealed by the quadratic form led to a more fruitful perspective.

3.2. N-to-Complex Conversion

Instead of focusing on the two unknown primes p_1 and p_2 , the analysis shifted toward their arithmetic mean,

$$m = \frac{p_1 + p_2}{2}.$$

Since $p_2 = 2m - p_1$, substitution into $N = p_1 p_2$ yields

$$N = p_1(2m - p_1),$$

which rearranges into the quadratic equation

$$p_1^2 - 2mp_1 + N = 0.$$

Applying the quadratic formula gives

$$p_1 = m \pm \sqrt{m^2 - N}.$$

While originally intended to study real-valued behavior such as the area under the quadratic curve, numerical experiments showed no stable correlation between m , the

discriminant, and the underlying semiprime N . This led to an alternative interpretation: whenever $m^2 < N$, the expression naturally produces complex-valued solutions. Reformulated, the roots become

$$p_1 = m \pm i\sqrt{N - m^2},$$

which suggests treating the quadratic structure as a mapping that associates each real input N with a complex value.

This observation led to the question: *What happens if the mean is fixed at a known value?* In particular, setting

$$m = \frac{1}{2}$$

aligns the construction with the critical line of the Riemann zeta function. With this choice, the expression becomes

$$p(N) = \frac{1}{2} \pm i\sqrt{N - \frac{1}{4}},$$

which places the real part of the resulting complex number exactly at $\Re(s) = \frac{1}{2}$.

This reformulation turns the semiprime-motivated quadratic identity into a simple but powerful real-to-complex conversion:

$$N \mapsto s(N) = \frac{1}{2} \pm i\sqrt{N - \frac{1}{4}}.$$

Thus, each real input N is mapped to a point on the critical line, with imaginary part determined by the square-root term. Numerical exploration of this mapping suggested a structured behavior in the corresponding evaluations of the Hardy Z -function, ultimately guiding the development of the valley-scanner algorithm presented in the following sections.

3.3. Valley Scanner Algorithm

The term *valley scanner* refers to the numerical procedure used to identify zeros of the Hardy function by detecting local minima of $|Z(t)|$ along a parameterized traversal of the critical line. The construction begins with the mapping

$$s = m \pm i\sqrt{N - m^2}, \quad m = \frac{1}{2},$$

so that

$$t = \Im(s) = \sqrt{N - m^2}, \quad N = t^2 + m^2.$$

Thus, increments in N generate corresponding increments in the imaginary part t , providing a simple mechanism for scanning $|Z(t)|$ across a continuous range of heights.

The objective is to evaluate $|Z(t)|$ for many successive values of N and to analyze the numerical landscape through direct observation and visualization.

A Python script was developed to perform the initial experimental exploration. The full, formatted version is available in the public repository: [playground.py](#). The listing below reproduces the exploratory code:

```

import subprocess
import sys

def ensure_lib(package):
    try:
        __import__(package)
    except ImportError:
        subprocess.check_call(
            [sys.executable, "-m", "pip", "install", package, "--quiet"]
        )

# Ensure required libraries
ensure_lib("mpmath")
ensure_lib("matplotlib")

import mpmath as mp
import matplotlib.pyplot as plt

mp.dps = 250 # precision

# From N, compute:
#  $s = m + \sqrt{N - m^2}i$ 
def N_to_complex(N, m):
    t = mp.sqrt(N - m ** 2)
    s = m + t * 1j
    return s, t

# Evaluate zeta
def Z(N, m):
    s, t = N_to_complex(N, m)
    z_val = mp.zeta(s)
    return t, z_val

dataset = []
m = mp.mpf('0.5') # mean = 1/2
for n in range(5000):
    t, z = Z(n, m)
    dataset.append([t.real, abs(z)])

ts = [t for t, _ in dataset]
zs = [z for _, z in dataset]

plt.figure(figsize=(10, 5))
plt.plot(ts, zs)
plt.xlabel("t (imaginary part of s)")
plt.ylabel("|Z(s)|")
plt.title("Z Test with N -> complex transformation (m = 1/2)")
plt.legend()
plt.grid(True)
plt.show()

```

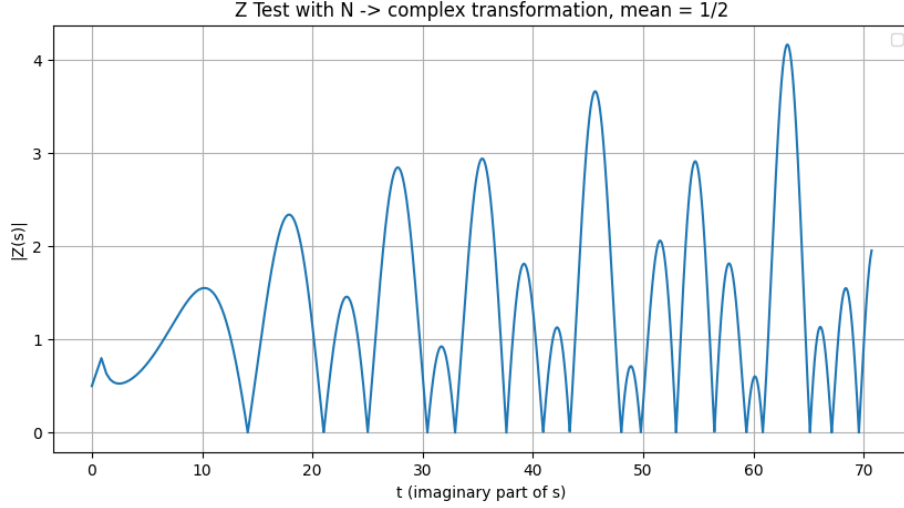



Figure 1. Illustration of the valley scanner detecting local minima of $|Z(s)|$.

A comparison with the first nontrivial zeros of the Riemann zeta function is shown in Table 1.

Table 1. First nontrivial zeros of the Riemann zeta function.

Index	Imaginary Part (t_n)
1	14.1347251417346937904572519835625
2	21.0220396387715549926284795938969
3	25.0108575801456887632137909925628
4	30.4248761258595132103118975305840
5	32.9350615877391896906623689640747
6	37.5861781588256712572177634807053
7	40.9187190121474951873981269146334
8	43.3270732809149995194961221654068
9	48.0051508811671597279424727494277
10	49.7738324776723021819167846785638
11	52.9703214777144606441472966088808
12	56.4462476970633948043677594767060
13	59.3470440026023530796536486749922
14	60.8317785246098098442599018245241
15	65.1125440480816066608750542531836
16	67.0798105294941737144788288965221
17	69.5464017111739792529268575265547

The mapping $N \mapsto s(N)$ consistently produces values of $\zeta(s)$ whose magnitude approaches zero whenever t is near a true zero of the zeta function. Increasing the resolution of N by refining decimal increments yields closer approximations of the zeros. Representative comparisons appear in Table 2.

Table 2. Comparison between computed and reference zeros of the Riemann zeta function.

N	Computed t	Reference t
200	14.13329402510257	14.13472514
442	21.017849556983705	21.02203964
626	25.014995502697975	25.01085758
926	30.426140077242792	30.42487613
1085	32.93554311074891	32.93506159
1413	37.586566749305526	37.58617816
1675	40.92370951	40.91871901
1877	43.32147273581544	43.32707328
2305	48.00781186432057	48.00515088
2478	49.77700272214067	49.77383248
2806	52.96933074902875	52.97032148
3186	56.442448564887755	56.44624770
3522	59.34433418617145	59.34704400
3701	60.83378995	60.83177853
4240	65.11336268386084	65.11254405
4500	67.08017591	67.07981053
4837	69.54674686856315	69.54640171

3.4. Identifying Zeros via Consecutive Minima of $|Z(t)|$

Although Python provides a flexible environment for numerical testing, the valley-scanning approach eventually becomes computationally intensive as t increases. To explore higher ranges of the critical line, optimized implementations were developed, including arbitrary-precision C++ code and parallel execution on AWS EC2.

The underlying idea remains unchanged: the magnitude $|Z(t)|$ exhibits alternating mountain-like peaks and valley-like minima. Zeros occur precisely at the bottoms of these valleys. The algorithm progressively scans increasing values of N (and corresponding t) to detect intervals where a transition from descent to ascent occurs.

Mountain walking and valley detection

The valley-scanner algorithm proceeds as follows:

- Choose an initial height t_0 .
- Convert t_0 to its corresponding real value N_0 using the mapping $N = t^2 + m^2$.
- Increment N by one unit (or by a chosen step size).
- Compute the corresponding t_1 and evaluate $|Z(t_1)|$.
- If $|Z(t_1)| < |Z(t_0)|$, the scan is descending; otherwise, it is ascending.
- A reversal of gradient (a “bounce”) indicates a local minimum and therefore a candidate zero.
- The candidate height is refined using a Newton root-finding method to obtain an accurate zero.

This mechanism forms the basis of the large-scale computations described in the subsequent sections.

Evaluation of the Hardy function

The numerical evaluation of $Z(t)$ in this work is based on the following components:

- For large t , the **Riemann–Siegel formula** is employed:

$$Z(t) = 2 \sum_{n=1}^N \frac{\cos(\theta(t) - t \log n)}{\sqrt{n}} + R(t),$$

where $N = \lfloor \sqrt{t/(2\pi)} \rfloor$, and $R(t)$ is the remainder term evaluated using Gabcke-type refinements.

- The phase function $\theta(t)$ is computed via a high-precision approximation of $\log \Gamma(\frac{1}{4} + \frac{it}{2})$, using Stirling expansions to avoid overflow:

$$\theta(t) = \text{Im} \left[\log \Gamma \left(\frac{1}{4} + \frac{it}{2} \right) \right] - \frac{t}{2} \log \pi.$$

- Each candidate zero obtained from the valley scanner is refined using a safeguarded Newton method with a Pegasus-type fallback (hybrid Newton–secant scheme). Computations are performed at 128–256 bits of precision, ensuring convergence even in steep or highly oscillatory regions.
- Validation is conducted by comparison with independently computed values (Odlyzko–Schönhage tables) and by a local consistency check (“ball test”) across neighboring zeros.

3.5. Parallelization and cloud execution

The dominant cost in evaluating $Z(t)$ via the Riemann–Siegel formula is the main summation

$$Z(t) = 2 \sum_{n=1}^N \frac{\cos(\theta(t) - t \log n)}{\sqrt{n}} + R(t),$$

where for fixed t , each term $\frac{\cos(\theta(t) - t \log n)}{\sqrt{n}}$ is independent of the others. This independence makes the summation an ideal target for parallelization.

We partition the summation range into subintervals and assign each to a separate thread:

$$S_i = \sum_{n=n_i}^{n_{i+1}-1} \frac{\cos(\theta(t) - t \log n)}{\sqrt{n}},$$

then perform a reduction step to aggregate all partial sums into the final value of $Z(t)$. The remainder term $R(t)$, corrected via the Gabcke [3] expansion, is evaluated separately.

Only the main summation domain is parallelized; the correction term remains serial. In practice, this allows nearly linear scaling with the number of available CPU cores. For cloud execution, AWS EC2 instances are configured to distribute the work across all physical cores using OpenMP. Each thread processes its assigned range of n -values

with a reduction clause for accumulation, yielding deterministic and numerically stable results.

The parallelized component is thus

$$\sum_{n=1}^N \frac{\cos(\theta(t) - t \log n)}{\sqrt{n}},$$

with ranges of n processed concurrently. This approach remains efficient for very large t , where $N = \sqrt{t/(2\pi)}$ may reach into the millions.

3.6. Why spacing heuristics (and naive prefiltering) can skip zeros

The zero-counting function on the critical line satisfies

$$N(T) = \frac{T}{2\pi} \log\left(\frac{T}{2\pi}\right) - \frac{T}{2\pi} + O(\log T), \quad (1)$$

which implies an average zero density $N'(T) \approx \frac{1}{2\pi} \log\left(\frac{T}{2\pi}\right)$ and hence an approximate average spacing

$$s_{\text{avg}}(T) \approx \frac{2\pi}{\log\left(\frac{T}{2\pi}\right)}. \quad (2)$$

3.6.0.1. Key caveat.. While (2) is a good mean predictor, the *local* spacing between consecutive zeros can deviate substantially. If one advances t by a deterministic step of size $s_{\text{avg}}(T)$, or discards points based on a naive threshold on $|Z(\frac{1}{2} + it)|$, genuine zeros can be missed.

3.6.0.2. Concrete example (skipped zero).. Consider two consecutive zeros (as verified against high-precision tables) near

$$t_1 = 30607946001.041439, \quad t_2 = 30607946001.175073.$$

Their observed gap is

$$\Delta_{\text{true}} = t_2 - t_1 \approx 0.133634.$$

The average-spacing prediction at $T \approx t_1$ from (2) is

$$s_{\text{avg}}(t_1) = \frac{2\pi}{\log\left(\frac{t_1}{2\pi}\right)} \approx 0.281673.$$

Stepping from t_1 by this predicted value leads to

$$t_1 + s_{\text{avg}}(t_1) \approx 30607946001.323112,$$

which overshoots the actual next zero at $t_2 \approx 30607946001.175073$. This illustrates explicitly that spacing-only stepping can skip zeros.

3.6.0.3. Implication for the valley scanner.. Early versions of the algorithm also experimented with rejecting points where $|Z(t)|$ exceeded a fixed threshold prior to refinement. In regions with sharply asymmetric or narrow valleys, this approach discarded valid neighborhoods and reduced recall. The final version of the valley scanner therefore avoids spacing-only jumps or aggressive prefilters. Instead, it uses dense local sampling with minima detection, followed by robust bracketing and root refinement (safeguarded Newton–secant), to prevent missed zeros.

3.6.0.4. Why GPU acceleration is not used.. Although the Riemann–Siegel summation is parallel in principle, the large- t computations here require high-precision arithmetic (typically 200–500 bits), which is not natively supported by current GPU hardware. CUDA cores are optimized for fixed-width types (such as `float32` and `float64`), whereas arbitrary precision is provided in this project by the MPFR and MPC libraries.

Emulating multiprecision arithmetic on GPUs would require splitting mantissas across multiple registers and synchronizing carries among threads, introducing substantial overhead and largely negating the theoretical speed-up. Moreover, the Riemann–Siegel evaluation involves nontrivial branching, logarithmic and trigonometric functions, and precision adjustments that are ill-suited to the SIMD-style execution model of GPUs.

For these reasons, high-precision evaluations of $Z(t)$ are performed on multi-core CPU instances (e.g. `c7i.8xlarge`) using OpenMP to parallelize the main summation. This CPU-based strategy offers deterministic reproducibility, robust numerical stability, and straightforward integration with the MPFR/MPC environment, while still achieving significant parallel speedups proportional to the number of cores.

Software and computational stack

The full computational workflow is built on the following components:

- **C++** with **MPFR/MPC** for the core high-precision evaluation of $Z(t)$.
- **AWS EC2** for high-performance compute instances with many CPU cores for parallel summations.
- **AWS Lambda** for orchestration of batch computations, job monitoring, and interaction with storage.
- **AWS S3** for persistent storage of logs, intermediate data, and zero tables.
- **AWS Cognito** for secure authentication to the web interface.
- **Docker** and **AWS ECR** to containerize the computation and the web frontend (React-based), enabling reproducible deployments.

A web application, accessible upon authorization, provides an interface for running valley scans and visualizing zeros up to heights on the order of 10^{16} . While the core computation source code is not yet fully publicly archived, all numerical datasets used in this work are available via the Zenodo and GitHub repositories referenced in the introduction. A public Docker image is also provided to allow local reproduction of the computations; see the project documentation for details: [README.md](#).

4. Results

The following tables present representative results obtained by the valley scanner across several ranges of t , including large heights where refinement is particularly challenging. The accompanying analysis of average zero spacing and local fluctuations was cross-checked by independent numerical scripts and symbolic tools to ensure robustness.

4.0.0.1. Scope of datasets.. The datasets shown here are intentionally concise and primarily illustrative. The valley-scanner framework is designed to detect zeros within any specified range of t ; the batch-processing architecture supports large-scale computation over much broader intervals, with feasibility limited mainly by the practical cost of cloud compute resources.

Table 3. Zero heights near $t \approx 1.1223 \times 10^9$.

t values
1122334455.05585408179597553373792753809670000283010284579445795581
1122334455.76031733677095186350835049571870227573769745536211492284
1122334455.96549341079558436082594904200898020740502977379833207345
1122334456.55122003455096567518805961663152718628678713782068430266
1122334456.69117656282847361682640086578613693696115124531479812624
1122334456.83867869196755030364329563494267434476218915992393583291
1122334457.15371418683945191486723066707952951489968507756091862973
1122334457.51390511989118717254611539418981231215406357632944032390
1122334457.76838741510620113455239734916962707865127304851980893696
1122334458.27155512243026569374168652041355212655972394951814858601
1122334458.44139136841692632939134894613526508309356307629078128927
1122334458.89501425212585279215913084722248689346999249199510723479
1122334458.95374621822277785558114513758006054444953092856654909690
1122334459.47180990048612714347328799643237077235245383445928401233
1122334459.78629512841632863716858891532650291270784909335852103315
1122334460.10222132816663933236459832453382800744621284674207091481
1122334460.34922785530398779220541441366240746601424248348045312059
1122334460.78924267475219010097101478269117727805242937830818188169
1122334461.03129979223085870682939517578522137411650684496149763174
1122334461.34105411839180789869245630301714029644280234966395722460
1122334461.68097513340590425015663066588174538735332969362229273055
1122334462.05653607553267581154859487598541969417453119764160088476
1122334462.35999091678192331701334666903361418145334941591973792237
1122334462.71045060502215878868517734546324575820479730409902219700
1122334463.22570175761036362585961260605296584140311386148887315924

Full dataset:

[refined_sample_1122334455.csv](#)

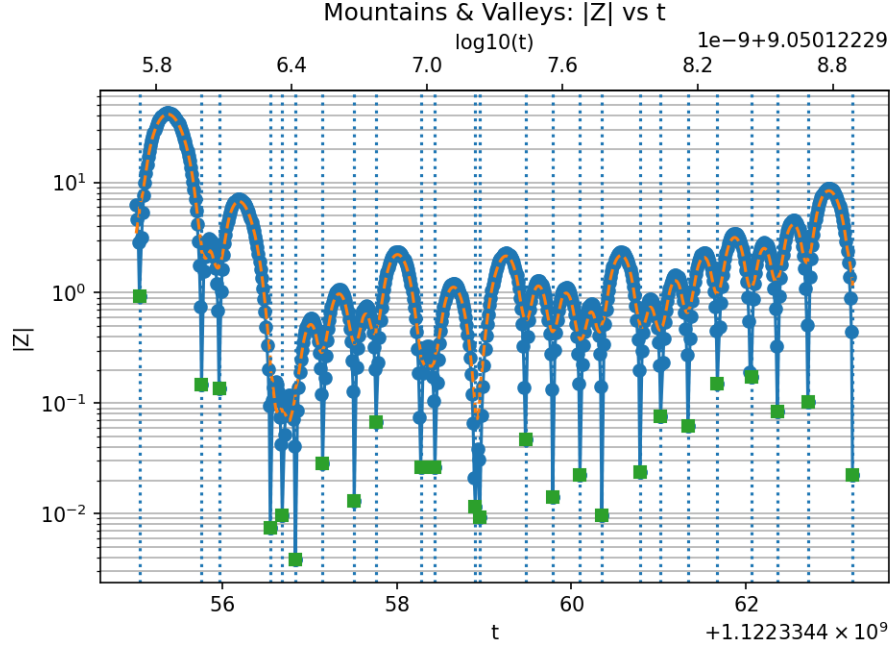


Figure 2. Valley-scanner output near $t \approx 1.1 \times 10^9$; local minima of $|Z(t)|$ are marked prior to refinement.

Table 4. Observed zero spacing vs. theoretical prediction at $t \approx 1.1223 \times 10^9$.

Parameter	Value / Description
Mean height T	1.122334459×10^9
Theoretical formula	$\Delta_{\text{avg}} \approx \frac{2\pi}{\log(T/2\pi)}$
$\log(T/2\pi)$	19.84
Predicted Δ_{avg}	0.33068
Observed mean(Δt)	0.34041
Relative difference	$\approx 2.94\%$

The observed mean spacing differs from the theoretical prediction by about 2.9%, which is consistent with normal local fluctuations at this height. This dataset includes twenty-four valid spacings (after discarding the initial $\Delta t = 0$), providing a stable estimate of the zero density.

4.0.0.2. Additional comments..

- The narrowest local gap ($\Delta t \approx 0.05873$) indicates a short clustering interval.
- The widest local gap ($\Delta t \approx 0.70446$) marks a mild sparsity region.
- Throughout this range, the valley scanner maintained $|Z(t)|$ residuals below 10^{-17} at refined zeros.

to formalize the $N \rightarrow \text{complex}$ conversion (with $m = \frac{1}{2}$) as a computational lens for studying the Hardy Z -function and its valley structure.

5.2. Exploration of alternative means

An important conceptual question is whether different choices of the mean parameter m might reveal structure away from the critical line. To probe this, exploratory computations were performed for several representative values:

$$m = \frac{1}{5}, \quad m = \frac{1}{\pi}, \quad m = \frac{1}{3}, \quad m = \frac{1}{e}.$$

In each case, the corresponding mapping $N \mapsto s(N) = m \pm i\sqrt{N - m^2}$ leads to vertical lines $\Re(s) = m$. Within the numerical ranges tested for these alternative means, no zeros were detected.

This does not constitute evidence against the existence (or nonexistence) of zeros off the critical line; the experiments were limited in scope and height. Rather, the outcome is consistent with the classical expectation that nontrivial zeros occur on $\Re(s) = \frac{1}{2}$, and it motivates further analytical work to understand why the $m = \frac{1}{2}$ case is especially well-aligned with the valley structure of $Z(t)$.

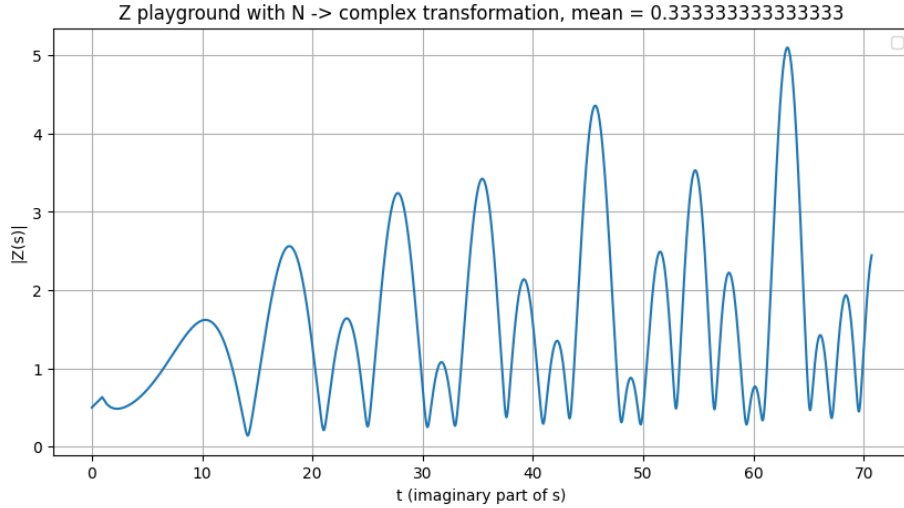


Figure 3. Valley-scan output with $m = 1/3$. No zeros were observed in the tested range.

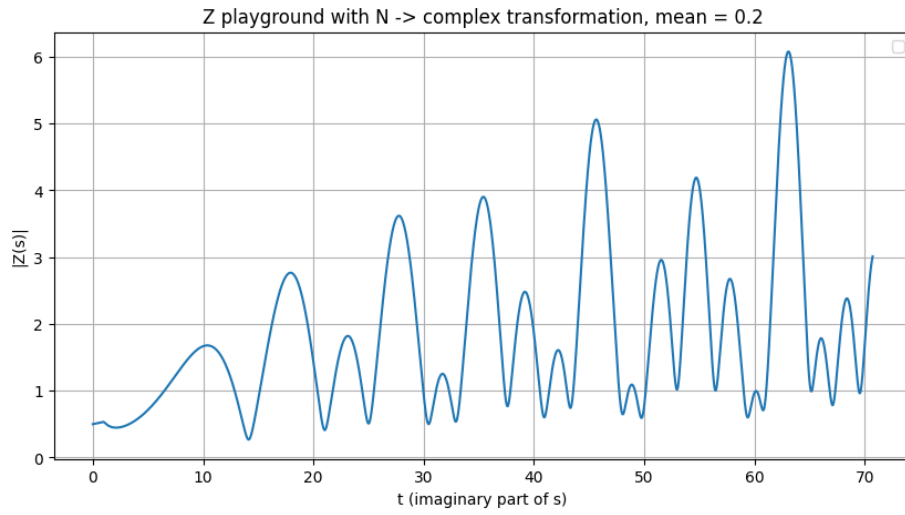


Figure 4. Valley-scan output with $m = 1/5$. No zeros were observed in the tested range.

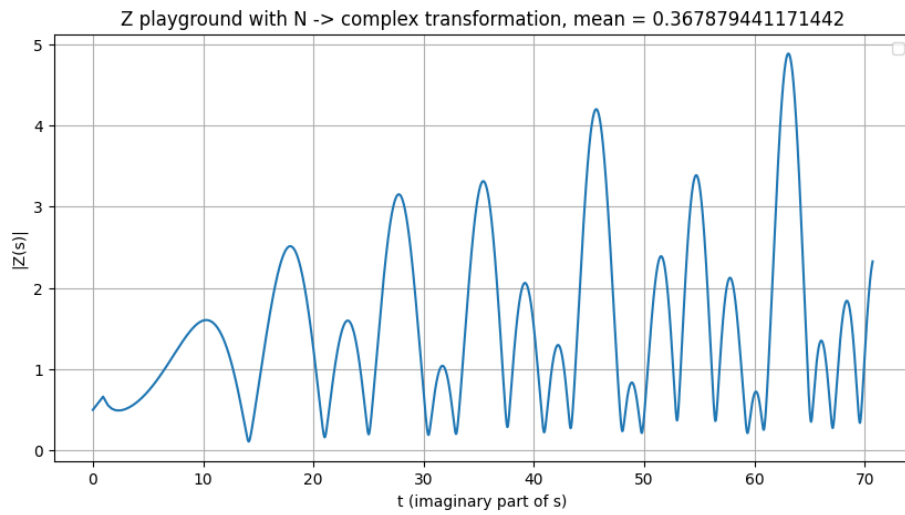


Figure 5. Valley-scan output with $m = 1/e$. No zeros were observed in the tested range.

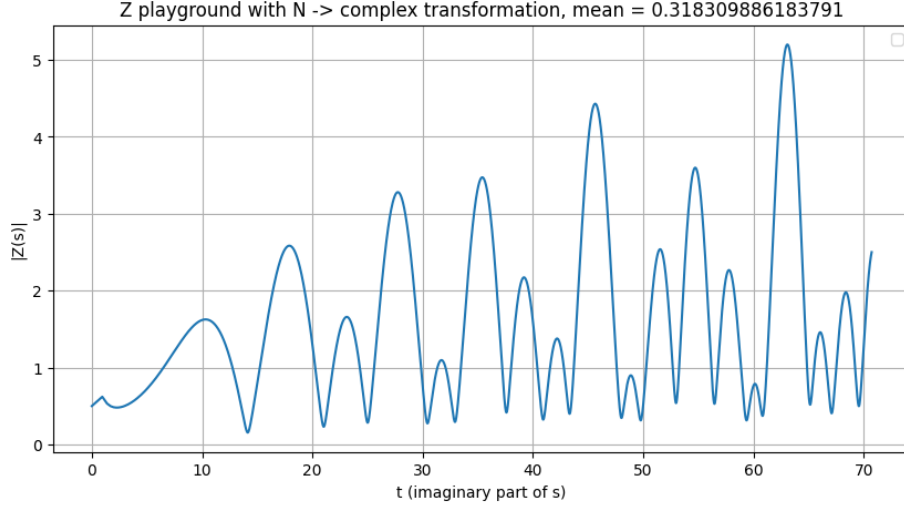


Figure 6. Valley-scan output with $m = 1/\pi$. No zeros were observed in the tested range.

6. Staircase representation of the explicit formula

We briefly describe a real-valued formulation of the correction term in the explicit formula for the Chebyshev function $\psi(x)$, based on partial sums over nontrivial zeros of the Riemann zeta function. This construction both motivates and mirrors the “mountain walk” intuition behind the valley scanner.

Let the nontrivial zeros of $\zeta(s)$ be

$$\rho_n = \frac{1}{2} + i\gamma_n, \quad \bar{\rho}_n = \frac{1}{2} - i\gamma_n, \quad \gamma_n > 0.$$

6.1. Staircase partial sums

For a fixed $x > 1$, define the complex partial sums

$$L_k(x) = \sum_{n=1}^k \frac{x^{\rho_n}}{\rho_n}, \quad k = 1, 2, \dots, \quad (3)$$

which correspond directly to the computational form

$$\text{term} = \frac{e^{\rho \log x}}{\rho}, \quad \text{correction} += \text{term}.$$

As k increases, these partial sums oscillate and gradually build up a staircase-like approximation to the full correction term appearing in the explicit formula for $\psi(x)$.

6.2. Including conjugate terms

Pairing each zero ρ_n with its conjugate $\bar{\rho}_n$ yields a real-valued staircase:

$$\tilde{L}_k(x) = \sum_{n=1}^k \left(\frac{x^{\rho_n}}{\rho_n} + \frac{x^{\bar{\rho}_n}}{\bar{\rho}_n} \right), \quad (4)$$

which matches the numerical implementation

$$\text{term} = \frac{e^{\rho \log x}}{\rho}, \quad \text{term2} = \frac{e^{\bar{\rho} \log x}}{\bar{\rho}}, \quad \text{correction} = \text{term} + \text{term2}.$$

6.3. Derivation of the real closed form

Set $L = \log x$. Then

$$x^\rho = x^{1/2} e^{i\gamma L}, \quad \frac{1}{\rho} = \frac{\frac{1}{2} - i\gamma}{\gamma^2 + \frac{1}{4}}.$$

A single conjugate pair contributes

$$\begin{aligned} \frac{x^\rho}{\rho} + \frac{x^{\bar{\rho}}}{\bar{\rho}} &= \frac{x^{1/2}}{\gamma^2 + \frac{1}{4}} \left[\left(\frac{1}{2} - i\gamma \right) e^{i\gamma L} + \left(\frac{1}{2} + i\gamma \right) e^{-i\gamma L} \right] \\ &= \frac{x^{1/2}}{\gamma^2 + \frac{1}{4}} \left[\frac{1}{2} (e^{i\gamma L} + e^{-i\gamma L}) + i\gamma (-e^{i\gamma L} + e^{-i\gamma L}) \right] \\ &= \frac{x^{1/2}}{\gamma^2 + \frac{1}{4}} \left[\cos(\gamma L) + 2\gamma \sin(\gamma L) \right]. \end{aligned}$$

Hence the real staircase (4) admits the fully real representation

$$\tilde{L}_k(x) = x^{1/2} \sum_{n=1}^k \frac{\cos(\gamma_n \log x) + 2\gamma_n \sin(\gamma_n \log x)}{\gamma_n^2 + \frac{1}{4}}. \quad (5)$$

6.4. Relation to the standard $-2\Re$ form

In the explicit formula it is common to write the correction term as

$$C_k(x) = -2\Re \sum_{n=1}^k \frac{x^{\rho_n}}{\rho_n}.$$

Using $\Re(z) = \frac{1}{2}(z + \bar{z})$, we find

$$C_k(x) = -\tilde{L}_k(x),$$

so (5) coincides with the standard explicit-formula correction up to an overall sign.

6.5. Real reformulation via $N_n = \gamma_n^2 + \frac{1}{4}$

Introduce the real quantity

$$N_n = \gamma_n^2 + \frac{1}{4},$$

which mirrors the “normal numbers” used in the numerical mountain walk. Then (5) becomes

$$\boxed{\tilde{L}_k(x) = x^{1/2} \sum_{n=1}^k \frac{\cos(\gamma_n \log x) + 2\gamma_n \sin(\gamma_n \log x)}{N_n}}. \quad (6)$$

In this form, the dependence on each zero enters only through the pair (γ_n, N_n) .

6.6. Computational remarks

Formula (6) eliminates the need to evaluate complex powers x^ρ . Each term now requires only real-valued operations: a single square root for $x^{1/2}$, one logarithm $\log x$, and calls to $\cos$ and $\sin$. Consequently, the staircase can be computed directly from the ordinates γ_n with good numerical stability and without the overhead of complex exponentiation.

In summary, pairing each zero with its conjugate yields the compact identity

$$\frac{x^\rho}{\rho} + \frac{x^{\bar{\rho}}}{\bar{\rho}} = \frac{x^{1/2} (\cos(\gamma \log x) + 2\gamma \sin(\gamma \log x))}{\gamma^2 + \frac{1}{4}},$$

which provides an efficient bridge between the complex explicit formula and a purely real numerical implementation.

A demonstration script implementing (6) is available at:
[staircase_with_real.py](#).

7. Conclusions

The main conclusions of this work can be summarized as follows:

- (1) The real-to-complex mapping

$$N \mapsto s(N) = \frac{1}{2} \pm i\sqrt{N - \frac{1}{4}},$$

combined with high-precision evaluation of the Hardy function via the Riemann–Siegel formula (with Gabcke-type [3] remainder) and safeguarded Newton refinement, successfully reproduces zeros of $\zeta(s)$ across a wide range of heights on the critical line, from classical low-lying zeros up to heights on the order of 10^{20} .

- (2) The numerical experiments indicate that this formulation offers a conceptually simple and effective framework for exploring the Riemann zeta function. The

- intrinsic symmetry about $\Re(s) = \frac{1}{2}$ is encoded directly in the mapping $N \mapsto s(N)$, and the valley structure of $|Z(t)|$ emerges naturally when scanning in N .
- (3) The staircase representation of the explicit formula, expressed purely in terms of the ordinates γ_n and real operations, provides an efficient and stable way to implement correction terms. This perspective is closely aligned with the valley-scanner approach, and may be extendable to other L -functions once appropriate analogues of the mapping and staircase are identified.
 - (4) While the present work does not attempt to resolve the Riemann Hypothesis, it offers a reproducible computational framework and a concrete set of high-precision datasets that may be useful for further numerical investigations of the zeta function and related objects.

8. Future work

- (1) The apparent symmetry of the “mountain” structures observed in $|Z(t)|$ suggests that a direct analysis of local maxima could be used to predict neighboring zeros. Preliminary experiments indicate that once a zero and its adjacent peak are located, the next zero can often be approximated by extrapolating the horizontal distance between the zero and the peak (approximately doubling this distance). A systematic study of this “peak-to-zero” geometry may reduce the computational effort required for large-scale searches.
- (2) A central motivation of this work is to encourage collaboration with specialists in analytic number theory to further develop the underlying analysis. The present results were obtained in an independent research setting, with limited resources, and rely heavily on numerical experimentation. Future progress—both in refining the method and in understanding its theoretical implications—would benefit from academic or institutional partnerships, including access to additional expertise and computing infrastructure.

Acknowledgments

The author acknowledges the use of modern artificial intelligence tools, specifically OpenAI’s ChatGPT models (GPT-4.5, GPT-5, and GPT-5.1), during the development of this work. These tools were employed to assist with technical language refinement, clarification of mathematical formulations, code review, dataset inspection, literature navigation, and verification of software architecture decisions.

All conceptual ideas, algorithms, experimental designs, and numerical results were developed, implemented, and validated by the author. AI systems served only as accelerators for tasks such as improving clarity of exposition, debugging syntax and code structure, and checking the consistency of numerical procedures.

The scientific contributions of this manuscript—including the real-to-complex parametrization, the valley-scanner framework, the computational experiments, and all interpretations of results—are entirely the work of the author. The preprint and associated software were prepared with human oversight at every stage.

The author gratefully recognizes the practical value of AI tools as companions for exploratory research in high-precision numerical computation, particularly in the absence of traditional peer feedback during early development.

Data and software availability

All source code, Docker images, and reproducibility datasets associated with this work are archived under the following Zenodo record:

DOI: [10.5281/zenodo.17566257](https://doi.org/10.5281/zenodo.17566257).

The corresponding GitHub repository and detailed instructions for running the valley scanner, web interface, and auxiliary scripts are linked from the Zenodo entry.

References

- [1] H. M. Edwards, *Riemann's Zeta Function*. Dover Publications, New York, 1974.
- [2] A. M. Odlyzko, Tables of zeros of the Riemann zeta function. Available at: https://www.dtc.umn.edu/~odlyzko/zeta_tables/
- [3] W. Gabcke, *Die Berechnung der Riemannschen ζ -Funktion mit Hilfe der Riemann–Siegel-Formel*. Dissertation, Universität Göttingen, 1979.
- [4] A. M. Turing, Some calculations of the Riemann zeta-function, *Proc. London Math. Soc. (3)* **4** (1953), 99–117.
- [5] A. M. Odlyzko and A. Schönhage, Fast algorithms for multiple evaluations of the Riemann zeta function, *Trans. Amer. Math. Soc.* **309** (1988), no. 2, 797–809.
- [6] J. M. Ball, A consistency test for computed zeros of the Riemann zeta function, *Math. Comp.* **57** (1991), no. 196, 837–846.
- [7] M. V. Berry and J. P. Keating, The Riemann zeros and eigenvalue asymptotics, *SIAM Rev.* **41** (1999), no. 2, 236–266.